

Lesson 8: Validation and Form Requests

□ Contents

1. [Introduction to Validation](#)
 2. [Basic Validation](#)
 3. [Validation Rules](#)
 4. [Custom Error Messages](#)
 5. [Form Request Classes](#)
 6. [Custom Validation Rules](#)
 7. [Conditional Validation](#)
 8. [Array Validation](#)
 9. [File Validation](#)
 10. [Practical Examples](#)
-

Introduction to Validation

What is Validation?

Validation = Verifying data correctness before processing or saving to database.

```
User Input → Validation → □ Valid → Process  
                    → □ Invalid → Show Errors
```

Why Validation is Important?

□ Protect database from incorrect data □ Improve user experience with clear messages □
Prevent security vulnerabilities □ Ensure data consistency

Basic Validation

In Controller

```
use Illuminate\Http\Request;

class PostController extends Controller
{
    public function store(Request $request)
    {
        $validated = $request->validate([
            'title' => 'required|max:255',
            'content' => 'required',
            'email' => 'required|email',
        ]);

        Post::create($validated);

        return redirect()->back()->with('success', 'Saved successfully');
    }
}
```

Display Errors in Blade

```
@if ($errors->any())
    <div class="alert alert-danger">
        <ul>
            @foreach ($errors->all() as $error)
                <li>{{ $error }}</li>
            @endforeach
        </ul>
    </div>
@endif

@error('title')
    <div class="alert alert-danger">{{ $message }}</div>
@enderror

<input type="text" name="title" value="{{ old('title') }}" class="@error('title')
is-invalid @enderror">
@error('title')
    <span class="invalid-feedback">{{ $message }}</span>
@enderror
```

Validation Rules

Basic Rules

```
$request->validate([
    'name' => 'required',
    'title' => 'required|string|max:255',
    'email' => 'required|email|unique:users,email',
    'age' => 'required|integer|min:18|max:100',
    'price' => 'required|numeric|min:0',
    'birth_date' => 'required|date',
    'published_at' => 'nullable|date|after:today',
    'agree' => 'required|accepted',
    'website' => 'nullable|url',
    'password' => 'required|string|min:8|confirmed',
]);
```

String Rules

```
'name' => ['required', 'string', 'min:3', 'max:255', 'alpha'],
'username' => 'required|string|regex:/^[a-zA-Z0-9_]+$/',
```

Number Rules

```
'age' => ['required', 'integer', 'min:18', 'max:100', 'between:18,65'],
'price' => 'required|numeric|min:0|max:999999.99',
'quantity' => 'required|integer|digits:4',
```

Date Rules

```
'birth_date' => ['required', 'date', 'before:today', 'after:2000-01-01'],
'start_date' => 'required|date',
'end_date' => 'required|date|after:start_date',
```

Database Rules

```
'email' => 'required|email|unique:users,email',
'email' => 'required|email|unique:users,email,' . $userId,
'category_id' => 'required|exists:categories,id',
```

File Rules

```
'avatar' => ['required', 'image', 'mimes:jpeg,png,jpg,gif', 'max:2048'],  
'document' => 'required|file|mimes:pdf,doc,docx|max:10240',
```

Custom Error Messages

```
$request->validate([  
    'title' => 'required|max:255',  
    'email' => 'required|email',  
], [  
    'title.required' => 'Post title is required',  
    'title.max' => 'Title must not exceed 255 characters',  
    'email.required' => 'Email is required',  
    'email.email' => 'Email is invalid',  
]);
```

Form Request Classes

Create Form Request

```
php artisan make:request StorePostRequest
```

app/Http/Requests/StorePostRequest.php:

```
<?php  
  
namespace App\Http\Requests;  
  
use Illuminate\Foundation\Http\FormRequest;  
  
class StorePostRequest extends FormRequest  
{  
    public function authorize(): bool  
    {  
        return true;  
    }  
}
```

```

    }

    public function rules(): array
    {
        return [
            'title' => 'required|string|max:255',
            'content' => 'required|string',
            'category_id' => 'required|exists:categories,id',
            'tags' => 'nullable|array',
            'tags.*' => 'exists:tags,id',
            'published_at' => 'nullable|date|after_or_equal:today',
        ];
    }

    public function messages(): array
    {
        return [
            'title.required' => 'Post title is required',
            'content.required' => 'Post content is required',
        ];
    }

    public function attributes(): array
    {
        return [
            'title' => 'post title',
            'content' => 'post content',
        ];
    }
}

```

Usage in Controller

```

use App\Http\Requests\StorePostRequest;

class PostController extends Controller
{
    public function store(StorePostRequest $request)
    {
        $validated = $request->validated();
        Post::create($validated);
        return redirect()->route('posts.index');
    }
}

```

Custom Validation Rules

Create Custom Rule

```
php artisan make:rule Uppercase
```

app/Rules/Uppercase.php:

```
<?php

namespace App\Rules;

use Closure;
use Illuminate\Contracts\Validation\ValidationRule;

class Uppercase implements ValidationRule
{
    public function validate(string $attribute, mixed $value, Closure $fail): void
    {
        if (strtoupper($value) !== $value) {
            $fail('The :attribute must be uppercase');
        }
    }
}
```

Usage

```
use App\Rules\Uppercase;

$request->validate([
    'code' => ['required', 'string', new Uppercase],
]);
```

Conditional Validation

```
public function rules(): array
```

```
{
    $rules = [
        'title' => 'required|string|max:255',
        'content' => 'required|string',
    ];

    if ($this->input('type') === 'video') {
        $rules['video_url'] = 'required|url';
        $rules['duration'] = 'required|integer';
    }

    return $rules;
}
```

Array Validation

```
$request->validate([
    'tags' => 'required|array|min:1|max:5',
    'tags.*' => 'string|max:50',

    'products' => 'required|array|min:1',
    'products.*.name' => 'required|string|max:255',
    'products.*.price' => 'required|numeric|min:0',
    'products.*.quantity' => 'required|integer|min:1',
]);
```

File Validation

```
$request->validate([
    'avatar' => 'required|image|max:2048',
    'document' => 'required|file|mimes:pdf,doc,docx',
    'avatar' => [
        'required',
        'image',
        'dimensions:min_width=100,min_height=100,max_width=1000,max_height=1000',
    ],

    'photos' => 'required|array|min:1|max:5',
```

```
'photos.*' => 'image|mimes:jpeg,png,jpg|max:2048',  
]);
```

Practical Examples

Example 1: User Registration

```
class StoreUserRequest extends FormRequest  
{  
    public function rules(): array  
    {  
        return [  
            'name' => 'required|string|max:255',  
            'email' => 'required|email|unique:users,email',  
            'password' => 'required|string|min:8|confirmed',  
            'phone' => 'nullable|string|regex:/^[0-9]{10}$/ ',  
            'birth_date' => 'required|date|before:today',  
            'avatar' => 'nullable|image|max:2048',  
            'agree_terms' => 'required|accepted',  
        ];  
    }  
}
```

Example 2: Post Creation

```
class StorePostRequest extends FormRequest  
{  
    public function rules(): array  
    {  
        return [  
            'title' => 'required|string|max:255',  
            'content' => 'required|string|min:100',  
            'category_id' => 'required|exists:categories,id',  
            'tags' => 'nullable|array|max:5',  
            'tags.*' => 'exists:tags,id',  
            'status' => 'required|in:draft,published',  
        ];  
    }  
}
```

Best Practices

- Use Form Requests for organization
 - Use validated() method only
 - Clear error messages
 - Custom Rules for complex logic
 - Remember old() in forms
 - Implement proper authorization()
 - Don't use all() instead of validated()
 - Don't forget to check authorization
 - Don't use weak validation rules
-

Next Step

After completing this lesson, you're ready for:

Lesson 9: File Upload and Storage

- File Upload
 - Storage Configuration
 - File Management
-

Happy Learning! □